

Binary Search

Binary Search is a searching algorithm used in a sorted array by repeatedly dividing the search interval in half and the correct interval to find is decided based on the searched value and the mid value of the interval.

Binary Search

	0	1	2	3	4	5	6	7	8	9
Search 23	2	5	8	12	16	23	38	56	72	91
	L=0	1	2	3	M=4	5	6	7	8	H=9
23 > 16 take 2 nd half	2	5	8	12	16	23	38	56	72	91
	0	1	2	3	4	L=5	6	M=7	8	H=9
23 < 56 take 1 st half	2	5	8	12	16	23	38	56	72	91
	0	1	2	3	4	L=5, M=5	H=6	7	8	9
Found 23, Return 5	2	5	8	12	16	23	38	56	72	91

Binary Search

```
// Return position of element in sorted
// array of size n with value K.
int binary(int array[], int n, int K) {
    int l = -1;
    int r = n; // l, r are beyond array bounds
    while (l+1 != r) { // Stop when l, r meet
        int i = (l+r)/2; // Check middle
        if (K < array[i]) r = i; // Left half
        if (K == array[i]) return i; // Found it
        if (K > array[i]) l = i; // Right half
    }
    return n; // Search value not in array
}
```

Binary Search - Best Case Time Complexity

Best Case Time Complexity of Binary Search

Algorithm: $O(1)$

*Best case is when the element is at the middle index of the array. It takes only one comparison to find the target element. So the best case complexity is **$O(1)$** .*

Binary Search - Average Case Time Complexity

Time Complexity Analysis:

Each step cuts the search space in half, so the number of elements checked follows this pattern:

Step	Remaining Elements
1	n
2	$n/2$
3	$n/4$
4	$n/8$
...	...
k	$n / 2^{k-1}$

We stop when there is **1 element left**, i.e.:

$$\frac{n}{2^k} = 1$$

Solve for k :

$$2^k = n \quad \Rightarrow \quad k = \log_2(n)$$

Binary Search - Worst Case Time Complexity

Worst Case Time Complexity of Binary Search

Algorithm: $O(\log N)$

The worst case will be when the element is present in the first position. As seen in the average case, the comparison required to reach the first element is $\log N$. So the time complexity for the worst case is $O(\log N)$.

Divide-and-Conquer Algorithm Design Paradigm

Divide-and-conquer.

- Break up problem into several parts.
- Solve each part recursively.
- Combine solutions to sub-problems into overall solution.

Most common usage.

- Break up problem of size n into **two** equal parts of size $\frac{1}{2}n$.
- Solve two parts recursively.
- Combine two solutions into overall solution in **linear time**.

Consequence.

- Brute force: n^2 .
- Divide-and-conquer: $n \log n$.

Divide et impera.
Veni, vidi, vici.
- Julius Caesar

Mergesort

Sorting

Sorting. Given n elements, rearrange in ascending order.

Applications.

- Sort a list of names.
- Organize an MP3 library.
- Display Google PageRank results.
- List RSS news items in reverse chronological order.

obvious applications

- Find the median.
- Find the closest pair.
- Binary search in a database.
- Identify statistical outliers.
- Find duplicates in a mailing list.

**problems become easy once
items are in sorted order**

- Data compression.
- Computer graphics.
- Computational biology.
- Supply chain management.
- Book recommendations on Amazon.
- Load balancing on a parallel computer.

non-obvious applications

...

Mergesort

Mergesort.

- Divide array into two halves.
- Recursively sort each half.
- Merge two halves to make sorted whole.



Jon von Neumann (1945)

A	L	G	O	R	I	T	H	M	S
---	---	---	---	---	---	---	---	---	---

A	L	G	O	R	I	T	H	M	S
---	---	---	---	---	---	---	---	---	---

divide $O(1)$

A	G	L	O	R	H	I	M	S	T
---	---	---	---	---	---	---	---	---	---

sort $2T(n/2)$

A	G	H	I	L	M	O	R	S	T
---	---	---	---	---	---	---	---	---	---

merge $O(n)$

Merging

Merging. Combine two pre-sorted lists into a sorted whole.

How to merge efficiently?

- Linear number of comparisons.
- Use temporary array.

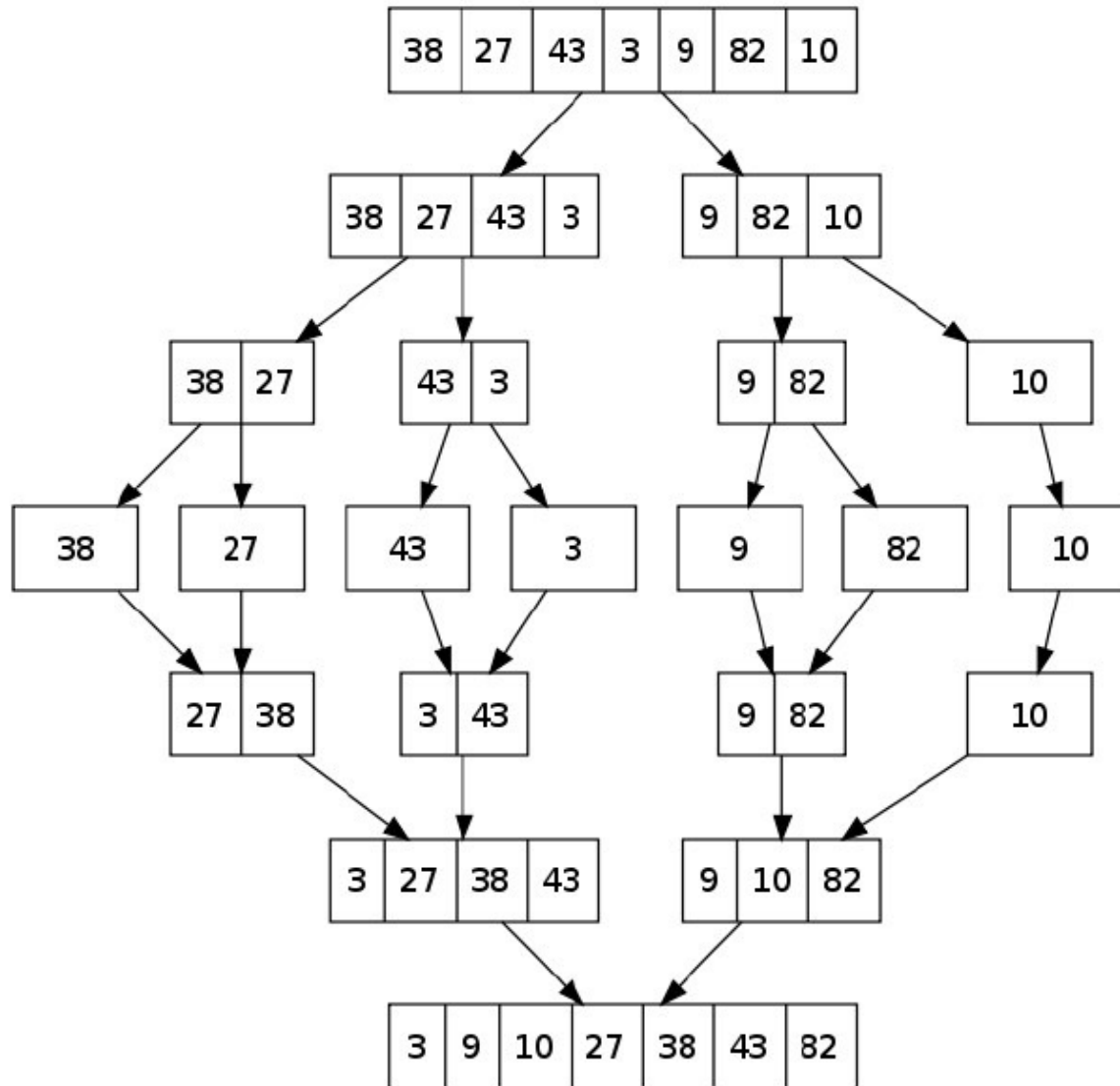


Challenge for the bored. In-place merge. [Kronrud, 1969]

↑
using only a constant amount of extra storage

DEMO
(05-demo-merge)

Merge Sort



A Useful Recurrence Relation

Def. $T(n)$ = number of comparisons to mergesort an input of size n .

Mergesort recurrence.

$$T(n) \leq \begin{cases} 0 & \text{if } n=1 \\ \underbrace{T(\lceil n/2 \rceil)}_{\text{solve left half}} + \underbrace{T(\lfloor n/2 \rfloor)}_{\text{solve right half}} + \underbrace{n}_{\text{merging}} & \text{otherwise} \end{cases}$$

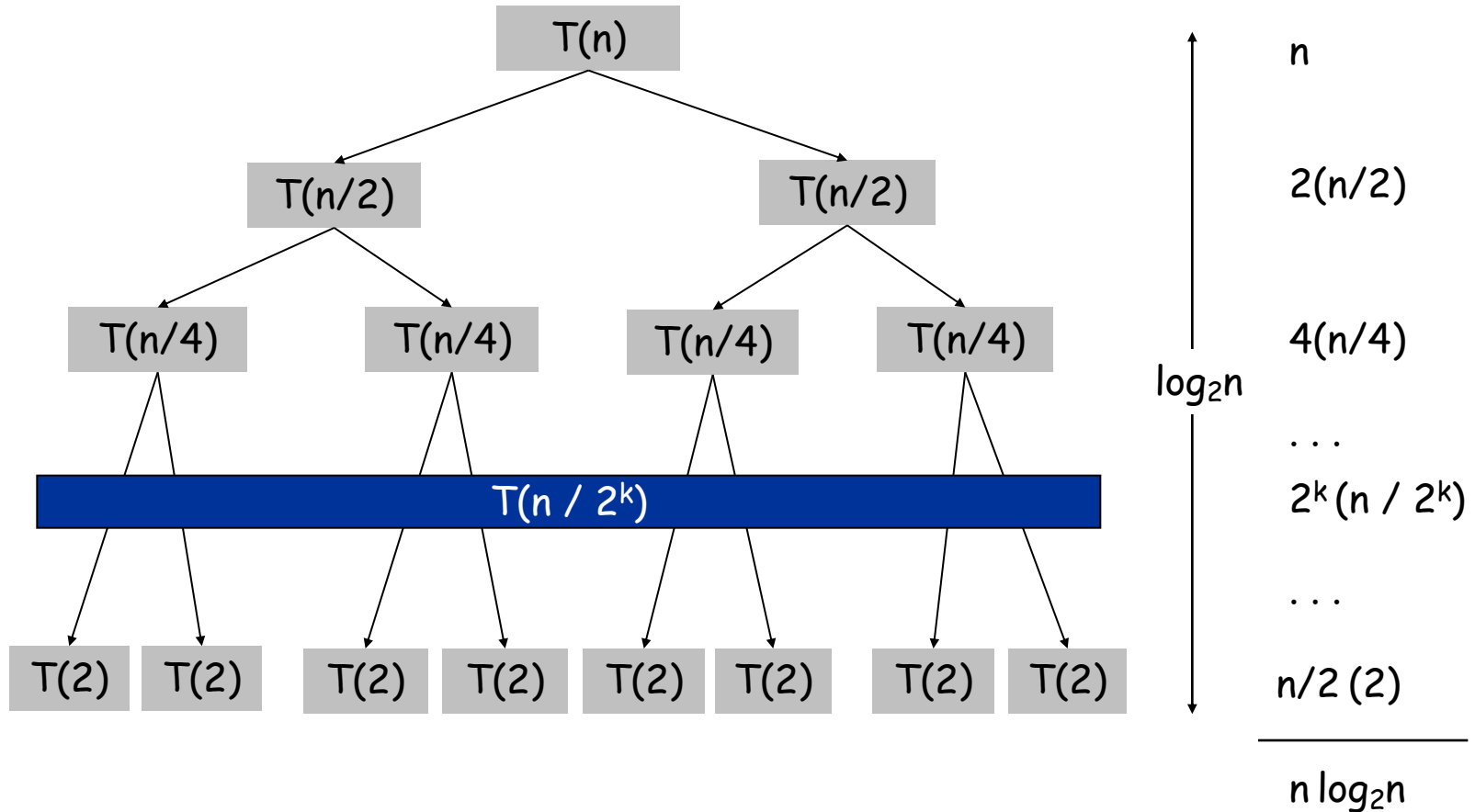
Solution. $T(n) = O(n \log_2 n)$.

Assorted proofs. We describe several ways to prove this recurrence. Initially we assume n is a power of 2 and replace $\leq$ with $=$.

Proof by Recursion Tree

$$T(n) = \begin{cases} 0 & \text{if } n=1 \\ \underbrace{2T(n/2)}_{\text{sorting both halves}} + \underbrace{n}_{\text{merging}} & \text{otherwise} \end{cases}$$

assumes n is a power of 2



Proof by Induction

Claim. If $T(n)$ satisfies this recurrence, then $T(n) = n \log_2 n$.

$$T(n) = \begin{cases} 0 & \text{if } n=1 \\ \underbrace{2T(n/2)}_{\text{sorting both halves}} + \underbrace{n}_{\text{merging}} & \text{otherwise} \end{cases}$$

↑
assumes n is a power of 2

Pf. (by induction on n)

- **Base case:** $n = 1$.
- **Inductive hypothesis:** $T(n) = n \log_2 n$.
- **Goal:** show that $T(2n) = 2n \log_2 (2n)$.

$$\begin{aligned} T(2n) &= 2T(n) + 2n \\ &= 2n \log_2 n + 2n \\ &= 2n(\log_2(2n) - 1) + 2n \\ &= 2n \log_2(2n) \end{aligned}$$